

Incorporating Gradients to Rules: Toward Online, Adaptive Provenance-Based Intrusion Detection

Zhenyuan Li , Lingzhi Wang , Zhengkai Wang , Xiangmin Shen , Haitao Xu , Yan Chen , *Fellow, IEEE*,
and Shouling Ji , *Member, IEEE*

Abstract—As cyber-attacks become increasingly sophisticated and stealthy, accurately distinguishing between benign behavior and malicious intrusions has become both more critical and more challenging. Provenance-based intrusion detection systems (PIDS) show strong potential for detecting malicious activities through fine-grained causality analysis, which has gained significant attention from both industry and academia. Among the various PIDS approaches, rule-based systems are particularly favored for their low overhead, real-time detection capability, and interpretability. However, these systems face challenges in reducing false positive rates, primarily due to the lack of fine-tuned rules and specific environments. In this paper, we introduce CAPTAIN+, a rule-based PIDS that autonomously adapts to diverse environments online. Specifically, we propose three adaptive parameters to adjust the detection configuration for nodes, edges, and alarm generation thresholds. Initially, we build a differentiable tag propagation framework and utilize the gradient descent algorithm to optimize these adaptive parameters based on the training data. In this extended version, we integrate an online learning module into the detection stage to dynamically optimize adaptive parameters based on real-time feedback from the detection process. We evaluate CAPTAIN+ based on data from DARPA TC, OpTC datasets, and PKU ASAL datasets. The results demonstrate that CAPTAIN+ offers superior detection accuracy, lower detection latency, reduced runtime overhead, long-term resilience against concept drift, and more interpretable detection results compared to state-of-the-art PIDS.

Index Terms—Intrusion detection, provenance graph analysis, online learning, adaptive learning.

I. INTRODUCTION

ADVANCED Persistent Threats (APTs) pose a significant risk to both government and industry [1]. For example, the 2017 Equifax data breach underscored critical privacy and security vulnerabilities [2], [3]. Moreover, attackers continuously evolve their tactics to infiltrate systems and remain undetected for extended periods. While provenance-based intrusion

detection systems (PIDS) have gained attention for their causal analysis capabilities, they still face key challenges in practical applications, including alarm fatigue, high operational overhead, and long detection latency [4], [5].

Alarm fatigue [6], [7] is a critical challenge in the cybersecurity industry. A recent survey [8] reports that large enterprises could receive over 100,000 alarms daily, with security analysts managing an overwhelming average of 5,000 alerts per day, most of which are false positives. The excessive volume of alarms can lead to serious consequences. For example, the damage from the 2013 Target data breach was exacerbated by a failure to respond to alerts on time. Alert overload and ambiguity make it difficult for analysts to identify threats [9] quickly. Consequently, an abundance of false alarms can be as detrimental as a single missed one. Beyond alarm fatigue, cybersecurity systems also face challenges related to runtime overhead and detection latency. High computational overhead restricts large-scale deployment of detection systems [10], while prolonged detection delays impede timely threat response and reduce analyst productivity.

In academia, an increasing number of PIDS are adopting embedding techniques, such as Word2Vec [11] and Graph2Vec [12], to encode system entities and events. These systems further utilize neural networks, including Graph Neural Network (GNN) [13], [14] and Recurrent Convolutional Neural Network (RCNN) [15], to analyze information flow and generate detection results, termed embedding-based PIDS. Although they have achieved significant advancements in detection performance, they still encounter several challenges: C1. High Computation Overhead: Processing embedded features with deep learning techniques [12], [15], leading to substantial computational overhead. Besides, caching models and embeddings significantly increase memory usage [10]. C2. Long Detection Latency: Many system designs rely on time windows [16] or log batching [11], [12], introducing additional detection delays. C3. Uninterpretable Results: Many systems [11], [17] only report anomaly behavior without attack semantics. The “black box” nature of models limits understanding of the detection, complicating alert handling.

On the other hand, a group of PIDS [18], [19], [20], [21], [22] operates based on human-defined mechanisms to generate representations for system entities, propagate information flow, and trigger alarms, referred to as rule-based PIDS. Rule-based PIDS use simple operations to generate semantically rich alerts, accurately locate suspicious events, and

Received 12 February 2025; revised 28 July 2025; accepted 15 September 2025. Date of publication 18 September 2025; date of current version 14 January 2026. (Zhenyuan Li, Lingzhi Wang, and Zhengkai Wang contributed equally to this work.) (Corresponding author: Zhenyuan Li.)

Zhenyuan Li, Zhengkai Wang, Haitao Xu, and Shouling Ji are with the School of Software Technology, Zhejiang University, Hangzhou 310058, China, and also with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310058, China (e-mail: lizhenyuan@zju.edu.cn; wangzhengkai@zju.edu.cn; haitaoxu@zju.edu.cn; sji@zju.edu.cn).

Lingzhi Wang, Xiangmin Shen, and Yan Chen are with the Department of Electrical Engineering and Computer Science, Northwestern University, Evanston, IL 60208-3118 USA (e-mail: lingzhiwang2025@u.northwestern.edu; xiangminshen2019@u.northwestern.edu; ychen@northwestern.edu).

Digital Object Identifier 10.1109/TDSC.2025.3611461

require low computing resources. Furthermore, some rule-based approaches support real-time data processing, which reduces storage requirements and enables fast responses with low latency. In practice, their fine-grained detection and efficient performance are particularly notable. Despite their strengths, rule-based PIDS face several challenges in real-world deployment: **C4. Fixed Rules:** Rule-based PIDS often rely on simplistic and universal rules (such as a node's confidence level [18], [19], [20] or general system parameters [18], [19], [20], [22]) to distinguish between benign and malicious graphs. However, these fixed rules can easily lead to a system that is too lax or too strict. **C5. Inflexible Detection Settings:** Existing methods fail to account for variations across different detection environments, as their detection policies remain unchanged across different datasets. **C6. Manual Optimization:** Even when some adjustments are possible, extensive manual effort is required to configure these systems, relying heavily on user expertise and manual analysis of environmental factors. **C7. Online Adaptation to Concept Drift:** Shifts in a system's primary behaviors can degrade the performance of detection models and algorithms [8], [23]. Thus, detection systems must dynamically adjust their parameters in response to environmental changes. However, existing rule-based PIDS typically lack such adaptation mechanisms.

To address these limitations, we need an automated configuration mechanism that enables rule-based PIDS to dynamically adjust parameters based on detection feedback. Building on CAPTAIN, introduced in our previous conference paper [24], this extended version presents CAPTAIN+, which enhances parameter optimization mechanisms to enable online adjustments based on false alarm feedback. This improvement further reduces false positives while enhancing adaptability to concept drift, enabling better long-term performance and maintainability. Specifically, CAPTAIN+ is designed to retain the advantages of traditional rule-based PIDS while autonomously acquiring optimal configurations. To this end, we introduce three adaptive parameters, which provide the system with greater flexibility. To enable their optimization, we construct a learning module and transform the rule-based system into a differentiable function by calculating and recording the gradients for each parameter. This allows for a two-stage optimization: **1) Offline Training:** Using a loss function, we apply the gradient descent algorithm to optimize the adaptive parameters based on training data. **2) Online Adaptation:** During the detection phase, the learning module promptly computes loss upon receiving feedback on false or missed alarms. It then immediately updates the parameters to enhance subsequent detection. As summarized in Table I, CAPTAIN+ builds upon the lightweight and adaptive foundation of CAPTAIN by introducing online learning and the ability to learn from false negatives.

We evaluate CAPTAIN+ across a variety of benign and APT attack scenarios, leveraging widely recognized public datasets [25] from the DARPA TC Project [26], OpTC datasets [27], and simulated environment datasets [28] collected in collaboration with a SOC. The evaluation results show that CAPTAIN+ reduces false alarms by over 98% ($60.18\times$) on average compared to traditional rule-based PIDS. Additionally, CAPTAIN+ achieves significantly higher detection accuracy while

TABLE I
COMPARISON OF PIDS

Methods	Light-Weight	Adaptive	Online Learning	FN Learning
Rule-based PIDS [18]–[20], [22]	✓	✗	✗	✗
Embedding-based PIDS [11], [13], [15], [16]	✗	✓	✗	✓
CAPTAIN [24]	✓	✓	✗	✗
CAPTAIN+	✓	✓	✓	✓

maintaining CPU usage below 15%, along with substantially lower memory consumption and latency compared to state-of-the-art embedding-based PIDS. CAPTAIN+ reduces false alarms by over 76% (4.2-fold improvement) on average compared to CAPTAIN and can detect more malicious events. In addition, CAPTAIN+ effectively copes with the concept drift problem and demonstrates higher stability in the detection process. Finally, we analyze selected scenarios to assess the explainability of the learned configurations. This paper makes the following contributions:

- We developed CAPTAIN+, a PIDS that adjusts its tag-propagation rules offline based on benign data and learns online using both benign and malicious feedback with an online learning module. This design equips CAPTAIN+ with the high accuracy and adaptability of learning-based systems, while ensuring long-term maintainability.
- We have parameterized our CAPTAIN+ with a differentiable tag propagation framework, enabling parameter tuning via gradient descent across various environments, which preserves the lightweight, low-latency, and fine-grained detection capabilities inherent to rule-based systems.
- We implemented CAPTAIN+ and evaluated it in various scenarios, including the DARPA TC, OpTC datasets, and PKU ASAL datasets. Results show that CAPTAIN+ adapts well to different environments with improved detection accuracy, reduced latency, lowered overhead, enhanced maintainability, and more interpretable results. To facilitate future research, we open-source our work at <https://anonymous.4open.science/r/CAPTAINplus-DA05>.

II. BACKGROUND AND RELATED WORK

A. Provenance-based Intrusion Detection

Recently, provenance-based intrusion detection has garnered significant attention from both industry and academia due to its powerful capabilities for correlation and causal analysis. In provenance analysis, system behaviors are represented as directed acyclic graphs, termed provenance graphs [29], [30]. In these graphs, nodes represent system entities, such as processes, files, sockets, pipes, and memory objects, while edges denote interactions between entities, including file access and remote host connections. Based on how entity information, data/control flow, and system behaviors are modeled and aggregated, we categorize PIDS into two major approaches, namely, embedding-based and rule-based.

1) *Embedding-based PIDS*: Embedding-based systems utilize numerical vectors to represent, propagate, and aggregate information about system entities, events, and data/control flows within a graph [11], [13], [15], [16]. For instance, PROVIDECTOR [31] selects rare paths based on historical event frequency to generate embedding vectors for sequence learning and anomaly detection, which requires processing the entire path for detection. UNICORN[23] embeds extracted histograms from the provenance graph and clusters the embedded graph sketches. While these systems can achieve higher detection accuracy, they often fail to provide contextual explanations for the alarms they generate. For instance, FLASH [11] identifies suspicious entities with causal links but lacks detailed attack semantics for those interactions. As a result, more manual effort is required to review the corresponding audit logs and initiate a response.

Furthermore, since these techniques typically require structured input, such as graphs or paths, embedding-based PIDS often use time windows or log batches, which introduces additional detection delays, a concept we term “buffer time” in this paper. Specifically, buffer time is the duration of the time window during which streaming logs are accumulated to form a graph structure for subsequent processing. Previous studies have implemented fixed-length windows based on time [13] or the number of events [11], while others [12] have used variable-length windows. In the worst case, after an attack is initiated, a PIDS must wait for the entire buffer time before beginning the detection process.

2) *Rule-based PIDS*: Rule-based systems [18], [19], [20], [22] use human-designed rules to map system entities and events to predefined semantic units (e.g., tags [18], [19], TTPs [20]), which are then propagated along the information flow to generate detection results. For example, POIROT [32] models APT detection as a graph pattern matching problem, where manually constructed query graphs are aligned with the provenance graph. Such systems have gained popularity due to their real-time processing capabilities and lightweight nature. However, they often suffer from low detection accuracy due to the absence of fine-grained rules and environment-specific configurations. Most rule-based PIDS [18], [19], [20] implement simple, universal rules that make them inflexible and unsuitable for adaptation to diverse environments. These systems typically rely on discrete classifications to characterize the properties of system entities (e.g., private or public) and apply the same rules to all nodes and events. As a result, they lack the granularity needed to accurately distinguish between similar graphs, often resulting in either overly lenient or overly strict rules that cause false alarms or missed alarms. We will further explore these challenges in §II-E.

In this paper, we build CAPTAIN+ using a rule-based detection method for the following reasons: 1) Rule-based detection is computationally efficient [18], [23], [32]. 2) Rule-based methods are suitable for real-time detection, as they can process event streams incrementally [30]. 3) Rule-based methods [18], [19], [21] provide greater explainability in the detection process and results compared to embedding-based methods [11], [14], [23]. 4) Rule-based methods are robust against mimicry attacks [33]. CAPTAIN+ makes a significant advancement over traditional

rule-based PIDS by systematically addressing the limitations that hinder their real-world application. Unlike systems that rely on simple, inflexible rules, CAPTAIN+ introduces a novel, mathematically complete framework for the automatic fine-tuning of tag initialization, propagation, and thresholds. This approach offers greater flexibility and precision, enabling more accurate detection and response to security threats.

B. Configuration as an Optimization Problem

Finding the optimal configuration for a rule-based PIDS can be formalized as an optimization problem, where the goal is to identify the best parameters θ in a parameter space to minimize or maximize an objective function $J(\theta)$. One common approach to solving this optimization problem is gradient descent [34]. It utilizes the gradient of the objective function concerning the parameters, $\nabla_{\theta} J(\theta)$, to update θ according to $\theta = \theta - l \cdot \nabla_{\theta} J(\theta)$, where l is the learning rate. Many variants have been proposed based on the gradient descent algorithm, including stochastic gradient descent [35], mini-batch gradient descent, Adagrad [36], and Adam [37]. We discuss more algorithms and their applicability to CAPTAIN+ in §V-A.

Although optimization problems and their gradient-based solutions play a significant role in quantitative science and engineering [38], they are rarely discussed in prior work on rule-based PIDS because of the following challenges. First, existing research does not clearly define the adjustable parameters in rule-based PIDS. Second, designing an objective function, along with calculating, updating, and saving the gradients needed in the gradient descent algorithm, are non-trivial challenges for a rule-based PIDS. In this paper, we introduce CAPTAIN+ to address these two identified gaps.

C. Online Learning

Traditional adaptive learning methods often rely on a pre-defined training dataset and use a specific algorithm to train models offline. However, concept drift [39] brings a challenge to their long-term effectiveness in real-world applications. Online learning, which continuously updates its predictions based on sequentially arriving data, iteratively refines the model’s predictive capabilities. This method has been effectively used to adjust to diverse detection environments and rapidly changing attack vectors [40], [41], achieving higher detection accuracy by adapting in real-time to evolving environments [42].

However, the key challenge of online learning lies in selecting appropriate training data for continuous learning. This challenge arises from the variability of data across the detection and training phases, which necessitates modifications to both the parameter adaptation strategy and the gradient calculations used in gradient descent optimization. In this paper, we integrate online learning into CAPTAIN+ and customize the parameter update mechanism to address this challenge. By leveraging online learning, CAPTAIN+ dynamically adjusts the parameters of nodes affected by tag propagation. This approach not only improves detection accuracy and reduces false positives but also enhances the system’s maintainability.

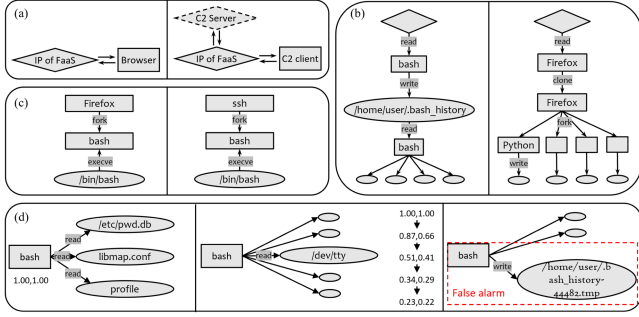


Fig. 1. Motivating Examples from Real-world Datasets

D. Threat Model and Assumptions

Following prior work [16], [18], [19], we assume OS kernels and auditing tools are part of the trusted computing base, meaning attackers cannot tamper with system audit data. We exclude hardware manipulation or other attacks that leave no trace in audit logs. Additionally, we assume the availability of audit logs reflecting daily activities without malicious behavior, for training in each detection environment.

E. Motivation Examples

This section presents several motivating examples to elucidate the challenges encountered by traditional, coarse-grained rule-based PIDS, as depicted in Fig. 1.

1) *“Grey” Nodes*: Recent reports indicate that attackers are increasingly exploiting commercial FaaS services, commonly used for legitimate purposes, to redirect traffic to Command and Control (C2) servers, as illustrated in Fig. 1(a). In such scenarios, both benign and malicious activities may involve communication with the same external FaaS IPs, making them difficult to differentiate. Existing rule-based PIDS often rely on overly coarse classifications when representing these IPs. For instance, SLEUTH [18] categorizes nodes into only three trustworthiness levels (Benign Authentic, Benign, and Unknown), which limits its ability to make precise distinctions. Consequently, these coarse-grained rules either generate excessive false alarms or fail to detect actual threats. A more fine-grained approach is needed to capture the “grey scale” of these IPs. Assigning a continuous trustworthiness value (e.g., a float between 0 and 1) to each node would enable more nuanced and adaptive detection.

2) *Dependency Explosion*: A dependency explosion occurs when the influence of a single event is propagated across an excessive number of system entities, resulting in high computational overhead and false alarms. Many approaches have been proposed to address this issue [43], [44], [45]. One category of solutions requires extensive instrumentation of applications or operating systems, which introduces significant overhead and limits practical deployment, particularly on end-host systems. Another category, found in rule-based PIDS, employs techniques such as event whitelisting [18], cost-based pruning, and stage-wise classification of suspicious events in an APT lifecycle [20]. For instance, MORSE [19] introduced decay and attenuation mechanisms to gradually reduce the maliciousness attributed to system entities over time.

However, these mechanisms apply uniform decay and attenuation factors to all nodes and edges, which leads to a trade-off between excessive false alarms and missed detections, as attackers can evade detection by forking numerous irrelevant processes or interacting with benign entities. Ideally, as illustrated in Fig. 1(b), dependency explosion should be contained in scenarios like the left graph while preserving maliciousness in cases like the right. To achieve finer-grained control, we propose assigning a customized propagation rate to each event. This allows maliciousness to dissipate rapidly for common events while persisting for anomalous ones, improving detection accuracy and reducing false positives.

3) *Customized Alarm Triggering*: In rule-based PIDS, alarms are triggered based on predefined conditions that are uniformly applied across all events, regardless of their context, which could result in suboptimal detection performance. As illustrated in Fig. 1(c), both processes Firefox and sshd frequently engage in network activities with various IP addresses. While sshd commonly executes scripts using bash, such behavior is unusual and potentially suspicious for Firefox. Despite similarities in event types and related entities, applying a uniform rule threshold for all “execute” events could either generate false alarms for sshd or fail to detect genuine threats involving Firefox. To address this challenge, a more context-aware approach is needed. For common events, a higher threshold should be set to minimize false alarms, whereas for rare events, a lower threshold is necessary to prevent missing true alarms. However, determining how to set distinct thresholds for the “execute bash” event for different processes remains challenging.

4) *Concept Drift*: The environment in which models operate is not static; changes in system behavior can lead to concept drift, resulting in degraded model performance. As illustrated in Fig. 1(d), when the process bash is introduced for the first time, both its initial confidentiality and reliability scores are set to the default value of 1. Over time, as more events are processed, tag propagation causes the tag scores of bash to change. Specifically, when bash executes a read operation on the file /dev/tty multiple times, its tag score significantly decreases. Similarly, when bash writes to the file /home/user/.bash_history-44482.tmp, an alarm is triggered because a low-reliability process is writing to a high-reliability file, which is identified as a false alarm.

These motivating examples highlight the challenges that current rule-based PIDS face in striking a balance between detection coverage and false positives, largely due to their coarse-grained rules. Furthermore, these systems often struggle with adaptability, as they cannot dynamically modify rules to suit an evolving environment. On the other hand, embedding-based PIDS, powered by deep learning, have the capability to generate tailored “rules” for each event and entity in the graph by mapping them into a feature space. However, the training and deployment of deep neural networks require substantial computational resources and storage, resulting in significant runtime overhead. Additionally, these systems typically lack transparency in their detection process and outcomes, making it

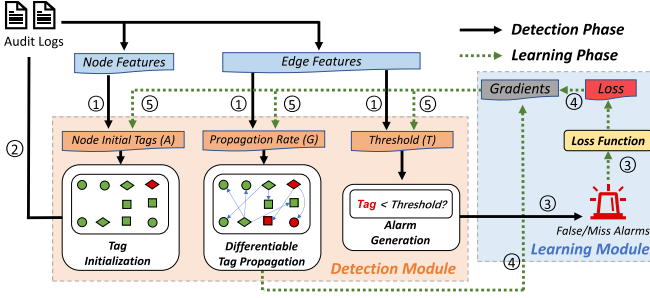


Fig. 2. CAPTAIN+'s Architecture

challenging to identify malicious behavior at the granularity of individual events.

To address these challenges, our approach enhances rule-based PIDS by incorporating fine-grained rules that achieve a better balance between detection accuracy and false positives. By enabling these systems to adapt and refine rules based on training data, we can significantly improve detection accuracy while avoiding the high computational costs and runtime overhead often associated with deep learning methods. Furthermore, to specifically address concept drift and enhance long-term maintainability, we incorporate an online learning module into CAPTAIN+'s detection phase. This allows for dynamic adjustments based on real-time feedback, which not only mitigates false alarms by adapting to evolving system behaviors but also reduces the system's reliance on static training datasets.

III. CAPTAIN+ DESIGN

As illustrated in Fig. 2, CAPTAIN+ consists of two primary components: the detection module and the learning module. The detection module processes audit logs to produce detection results. Concurrently, the learning module utilizes false alarms and missed alarms to fine-tune the detection module throughout the training and detection phases.

A. Adaptive Detection Module

As detailed in Algorithm 1 (lines 2-4) and Algorithm 2 (lines 1-4), the detection module in CAPTAIN+ operates in three sequential stages: tag initialization, tag propagation, and alarm generation. The process begins when new system entities are introduced; each is assigned an initial tag containing its security state and contextual information. Subsequently, as system events unfold, these tags propagate throughout the provenance graph, continuously updating the security context. In the final stage, the alarm generation component evaluates system activities against predefined security policies to identify and flag malicious behavior.

1) *Tag Design*: Inspired by previous provenance-based approaches that utilize data flow and control flow [46], CAPTAIN+ employs two types of tags: data tags and code tags as detailed in Table II. Data tags apply to all nodes within the provenance graph, quantifying the maliciousness of data through a numerical vector $\langle c, i \rangle$, where c represents the confidentiality score, and i the integrity score of the data. Conversely, code tags,

TABLE II
TAGS' DEFINITION IN CAPTAIN+

Tag Type		Value Range
Data Tag	Confidentiality (c)	0 (Most Confidential) to 1 (Public)
	Integrity (i)	0 (Lowest) to 1 (Highest)
Code Tag	Integrity (p)	0 (Lowest) to 1 (Highest)

Algorithm 1: Offline Learning.

```

Data: Streaming events  $S_E = \{e_i\}$ ;
Event parameters  $\mathbb{P} = \{A_i, G_i, T_i\}$ ; Learning rate  $lr$ ;
Number of epochs  $E$ ;
Parameters to update collection  $C_p \leftarrow \emptyset$ ;
Result: Updated parameters  $\mathbb{P}' = \{A'_i, G'_i, T'_i\}$ ;
1 for epoch From 1 To  $E$  do
  // Collect parameter to update
2   foreach  $e_i = (N_{si}, R_i, N_{di})$  in  $S_E$  do
3      $Tag_i \leftarrow \text{INITTAG}(e_i, A_i)$ ;
4      $\mathbb{A} \leftarrow \text{PROPTAG\&TRIGERALERT}(e_i, Tag_i, G_i, T_i)$ ;
5     if  $\mathbb{A}.\text{ISWRONG}()$  then
6        $grad_i \leftarrow \text{GRADFUNCION}(Tag_i, \mathbb{P})$ ;
7        $C_p[\mathbb{P}, grad_i] \leftarrow \text{PARAMETERCOLLECT}(\mathbb{P}, grad_i)$ ;
8     end
9   end
  // Update Parameter
10  foreach parameter  $\mathbb{P}$  in  $C_p$  do
11     $\mathbb{P}' \leftarrow \text{UPDATEPARAMETER}(\mathbb{P}, grad, lr)$ ;
12  end
13 end
14 return  $\mathbb{P}'$ 

```

indicated by $\langle p \rangle$, assess the integrity of the code and are exclusive to process nodes. All scores (c , i , and p) range from 0 to 1, where lower scores suggest potential compromise and higher scores indicate trustworthiness.

The tag system in CAPTAIN+ is modeled after previous rule-based PIDS methodologies [18], [19], though this paper does not delve into the tag design rationale. We establish that any new tag type integrated into CAPTAIN+ must meet three criteria: 1) numerical representation, 2) update via arithmetic calculations, and 3) alarm triggering based on tag values. For instance, an ‘‘exploitability’’ tag could be developed for web service nodes, complete with specific propagation and alarm rules. The focus here is on demonstrating the enhancements brought about by CAPTAIN+'s learning module rather than the specifics of tag design, leaving room for tailored tag development in particular scenarios.

The online learning design of CAPTAIN+ is built upon three adaptive parameters: 1) *Tag Initialization Value (A)* determines the initial tags for system entities. 2) *Tag Propagation Rate (G)* adjusts how system events influence tag values. 3) *Alarm Generation Threshold (T)* modifies the rules for alarm generation. We detail how CAPTAIN+ utilizes these parameters to enhance the flexibility of rule-based detection.

2) *Tag Initialization*: Proper tag initialization rules are pivotal as they determine the initial security states of system entities. For example, assigning a low integrity tag to a socket connected to a benign IP address could generate numerous false alarms.

Algorithm 2: Online Detection and Learning.

Data: Streaming events $S_E = \{e_i\}$;
Event parameters $\mathbb{P} = \{A_i, G_i, T_i\}$; **Learning rate** lr ;
Result: Alert $\mathbb{A}$; **Updated parameters** $\mathbb{P}' = \{A'_i, G'_i, T'_i\}$

```

1 foreach  $e_i = (N_{si}, R_i, N_{di})$  in  $S_E$  do
    // Detection Process
2    $Tag_i \leftarrow \text{INITTAG}(e_i, A_i)$ ;
3    $\mathbb{A} \leftarrow \text{PROPTAG\&TRIGERALERT}(e_i, Tag_i, G_i, T_i)$ ;
4   return ( $\mathbb{A}$ );
5   if  $\mathbb{A}.\text{ISWRONG}()$  then
       // Update Parameter
6      $lr' \leftarrow \text{ADJUST}(lr, \mathbb{A}.\text{CHECK}())$ ;
7      $grad \leftarrow \text{GRADFUNCION}(Tag_i)$ ;
8      $\mathbb{P}' \leftarrow \text{UPDATEPARAMETER}(\mathbb{P}, grad, lr')$ ;
       // Update Tags
9      $\text{UPDATETAGVALUE}(Tag_i, lr')$ 
10  end
11 end

```

This aspect, however, is often under-discussed in the literature [18], [19], where initial tags are typically assigned based on domain knowledge without subsequent adjustments. While MORSE acknowledges that tag initialization policies could be informed by previous tracing data, it lacks a detailed discussion or a practical methodology.

In CAPTAIN+, tags are initialized when nodes are first added to provenance graphs, as depicted in Fig. 2 at step ①. The detection system scans incoming system audit logs to identify new entities, which are then added as nodes with initial data tags. For new process nodes, we do not initialize code tags and data tags directly. A process node's code tag value is inherited from the data tag of the libraries loaded by it, and its data tag value is inherited from its parent nodes. For non-process nodes, we employ an adaptive parameter A at the node level to enable fine-grained tag initialization rules for any node $n_i \in N$ within the provenance graph. A stores its initial data tag based on the node's specific features, such as file path, process name, and command line. For instance, sockets with identical IP addresses receive the same initial data tag based on this feature.

The adaptive parameter A allows CAPTAIN+ to learn fine-grained tag initialization rules at the node level. CAPTAIN+'s learning module refines A through a training process, detailed in §III-B, beginning with a default value A_0 . To avoid missing threats early on, A_0 is conservatively set during training on benign data, accepting some initial false alarms. Unlike MORSE which uses conservative settings to capture all potential threats [19], CAPTAIN+ applies such settings only during the initial training phase. For example, we assign all IP addresses an initial integrity value of 0, ensuring that we do not miss any attackers entering through network communications.

A key challenge in provenance-based detection is establishing initial trust scores for system entities. Deriving these purely from data can be unreliable without extensive, domain-specific knowledge. To address this, CAPTAIN+ employs a principled, two-stage approach for score initialization and adaptation. First, during system deployment, an administrator performs a one-time manual configuration rule to set baseline confidentiality scores for different categories of entities (e.g., executable files, network sockets). This step injects essential domain knowledge

into the system from the outset. From that point forward, the system is fully automatic. When new entities are created during runtime, CAPTAIN+ instantly assigns them an initial score based on the pre-configured rules. These scores are then dynamically adjusted through both tag propagation and a continuous online learning process. In this latter process, an adaptive parameter, A , is refined based on real-time feedback to enhance future detection accuracy.

3) *Tag Propagation:* As depicted in Fig. 2 at step ②, once tags are initialized, specific system events trigger the propagation of these tags along the information flow, altering code and data tags accordingly. In CAPTAIN+, tag propagation is designed to transfer and accumulate potentially malicious intentions through tag values. For instance, the event (Firefox, read, IP) updates the data tag of Firefox to the lower of the two values between Firefox and IP, ensuring that any malicious intent carried in the information flow is preserved. Our propagation rules, detailed in Table III, are based on prior work [18], [19]. To ensure a fair comparison, we adopted the default hyperparameter values recommended by MORSE. These parameters, defined in Table III, include key terms such as data and code tags ($dtag, ptag$), quiescent tag values (T_{qb}, T_{qe} , decay rates (d_b, d_e) and attenuation factors (a_b, a_e).

However, such a mechanism can lead to a dependence explosion problem [19], resulting in excessive false alarms. For example, as discussed in §II-E, the event (bash, write, /home/user/.bash_history) commonly appears in many false alarm propagation chains, indicating widespread misclassification of benign activities as malicious within the provenance graph. To address this issue, we introduce an adaptive parameter G , which controls the propagation rate. Specifically, for any edge $e \in E$ characterized by ($src_node_feature, event_type, dest_node_feature$), G stores its propagation rate parameter $g_e \in [0, 1]$. During tag propagation, the destination node's tag tag_{dest} is updated as shown in Equation (1), where tag_{rule} typically equals tag_{src} , representing the tag value determined by the propagation rules listed in Table III. If g_e 's value is close to 0, it results in minimal changes to tag values, while a value close to 1 leads to significant changes.

$$tag_{dest}^{new} = g_e \cdot tag_{rule} + (1 - g_e) \cdot tag_{dest} \quad (1)$$

CAPTAIN+ continuously refines G through its online training process, with details elaborated in §III-B. Initially, we conservatively set all g_e values to 1, ensuring that during the initial phase, tag propagation occurs without attenuation. This allows for the maximum possible dissemination of information throughout the graph. This strategy ensures that we retain a broader set of potentially suspicious nodes and paths during the training stage. By preventing the premature dampening of signals, we provide the model with richer data to learn from, enhancing its ability to capture complex threat patterns. For subsequent online detection, this initial setup allows the system to maintain maximum vigilance against unknown entities, as potential risk indicators are propagated without being weakened by an initially conservative parameter.

4) *Alarm Generation:* As depicted in Fig. 2 at step ③, upon the occurrence of an event, the system evaluates whether it

TABLE III
TAG PROPAGATION POLICIES

Event	Tag to update	New tag value for different subject types		
			suspect	suspect environment
$create(s, x)$	$x.dtag$	$s.dtag$	$s.dtag$	$s.dtag$
$read(s, x)$	$s.dtag$	$\min(s.dtag, x.dtag)$	$\min(s.dtag, x.dtag)$	$\min(s.dtag, x.dtag)$
$write(s, x)$	$x.dtag$	$\min(s.dtag + a_b, x.dtag)$	$\min(s.dtag, x.dtag)$	$\min(s.dtag + a_e, x.dtag)$
$load(s, x)$	$s.ptag$		$\min(s.ptag, x.itag)$	
	$s.dtag$		$\min(s.dtag, x.dtag)$	
$exec(s, x)$	$s.ptag$	$x.itag$	$\min(x.itag, susp_env)$	$x.itag$
	$s.dtag$	$\langle 1.0, 1.0 \rangle$	$\min(s.dtag, x.dtag)$	$\min(s.dtag, x.dtag)$
$inject(s, s')$	$s'.stag$		$\min(s'.stag, s.itag)$	
	$s'.dtag$		$\min(s.dtag, s'.dtag)$	
$periodically:$	$s.dtag$	$\max(s.dtag, d_b * s.dtag + (1 - d_b) * T_q b)$	no change	$\max(s.dtag, d_e * s.dtag + (1 - d_e) * T_q e)$

TABLE IV
ALARM GENERATION POLICIES

Name	Description	Operation(s)	Data integrity condition	Other conditions
MemExec	Prepare binary code for execution	$mmap(s, p), mprotect(s, p)$	$s.itag < 0.5$	$incl_exec(p)$
FileExec	Execute file-based malware	$exec(s, o), load(s, o)$	$s.itag < 0.5$	$s.ptag > 0.5$
Inject	Process injection	$inject(s, s')$	$s.itag < 0.5$	$s'.ptag > 0.5$
ChPerm	Prepare malware file for execution	$chmod(s, o, p)$	$s.itag < 0.5$	$incl_exec(p)$
Corrupt	Corrupt files	$write(s, o), mv(s, o), rm(s, o)$	$s.itag < 0.5$	-
Escalate	Privilege escalation	$any(s)$	$s.itag < 0.5$	changed userid
DataLeak	Confidential data leak	$write(s, o)$	$s.itag < 0.5$	$s.ctag < 0.5, socket(o)$
MalFileCreation	Ingress Tool Transfer	$create(s, o)$	$s.itag < 0.5$	$File(o)$

meets the criteria for triggering an alarm according to predefined alarm generation rules. Our alarm generation policies, detailed in Table IV, are adopted from MORSE [19], in which $incl_exec(p)$ means p contains the execution permission, and $socket(o)$ holds when o refers to a socket. In general, these rules are based on the event, the associated tags of the subject and object, and a defined threshold that can be fine-tuned to manage alarm frequency. For instance, a file corruption alarm is initiated if a process with a data integrity value below 0.5 writes to a normal file. Frequent false alarms from events like (`sshd`, `execve`, `bash`) during training may indicate the need to raise the threshold for such cases.

We introduce an adaptive parameter T to enable fine-grained adjustment of the alarm threshold. Specifically, for any edge $e \in E$ in the provenance graph, T stores its alarm threshold $thr_e \in [0, 1]$. For an edge e , the detection function $f(e)$ is defined in Equation (2) where tag is the relevant node tag on edge e . A low threshold thr_e tends to suppress alarms, whereas a high threshold tends to promote alarm generation, allowing CAPTAIN+ to dynamically adjust response sensitivity at the edge level. Like A and G , T is refined through a training process discussed in §III-B. Furthermore, during the online learning phase in detection, T is adjusted in real-time based on incoming data and event outcomes. Initially, we set default thresholds (T_0) neutrally, assigning $thr_i = 0.5$ for every edge e to neither universally suppress nor promote alarm triggering.

$$f(e) = tag - thr_e \Rightarrow \begin{cases} \text{malicious,} & \text{if } f(e) < 0 \\ \text{benign,} & \text{otherwise} \end{cases} \quad (2)$$

It is noteworthy that CAPTAIN+'s methodology is systematic and independent of specific rules or tags, making it broadly

applicable to various rule-based PIDS, especially taint-analysis systems like SLEUTH [18] and HOLMES [20].

B. Configuration Learning Module

With the three adaptive parameters defined, the problem of adaptive configuration for the detection system is transformed into an optimization problem: finding the optimal values for A , G , and T that minimize a given objective function. This section details how we define the objective function, calculate the necessary gradients, and search for optimal parameter values. Compared to the previous CAPTAIN design, CAPTAIN+ incorporates online learning capabilities in addition to offline training to further enhance detection accuracy and long-term maintainability. The inclusion of an online learning mechanism allows CAPTAIN+ to continuously analyze and adapt to not only false positives but also false negatives, thereby significantly improving the reliability of the model.

1) *Loss Function*: The first step in optimization is defining the objective function, termed the loss function in our learning tasks. The goal of the learning module is to adjust parameters in a way that reduces classification errors (both false positives and false negatives) while preserving sensitivity to genuine malicious events. Consequently, the loss function consists of two components: a term for these errors ($\sum_{e \in E} \mathcal{L}(e)$) and a regularization term (R) to maintain generalization capabilities.

The error term, as shown in Equation (3), is designed to penalize false alarms — both false positives and false negatives. Specifically, for an event e , a penalty is incurred if the sign of its score $f(e)$ is incorrect. For a malicious event misclassified as benign (a false negative), its score $f(e)$ will be positive. The function in Equation (3) is designed to penalize the incorrect

score. Importantly, correctly classified events are excluded from the loss computation to avoid unnecessary updates that may reduce sensitivity or increase false alarms.

$$\mathcal{L}(e) = (1 + |f(e)|)^2 - 1 \quad (3)$$

The second component of our loss function is the regularizer term. As previously mentioned, CAPTAIN+'s offline training stage primarily relies on benign data to learn normal behaviors within the detection environment. In the field of machine learning, One-class classification (OCC) algorithms address scenarios where only one class of samples, typically the "normal" class, is available during training [47]. Drawing inspiration from seminal works in this area [48], [49], we incorporate a regularization term into our loss function to prevent the parameters from becoming overly permissive in the absence of malicious training samples. The core idea is to make the adaptive parameters as close as possible to the default values (A_0, G_0, T_0). By minimizing the l_2 distance between the adaptive parameters and the default values, we protect the detection capability during training, as shown in Equation (4), where α, γ , and τ are the regularizer coefficients. In the evaluation section, we discuss the effect of the regularizer coefficients (§IV-D2) and methods used to fine-tune them (§IV-A3).

$$\text{Loss} = \sum_{e \in E} \mathcal{L}(e) + \underbrace{\alpha \|A - A_0\|_2 + \gamma \|G - G_0\|_2 + \tau \|T - T_0\|_2}_{\text{Offline Training}} \quad (4)$$

Unlike offline training, online learning does not incorporate regularization. This omission is deliberate because the errors identified in the detection phase need immediate and effective updates. The regularization techniques used during training to prevent overfitting are unsuitable for online learning, which requires the system to respond swiftly to both false positives and negatives without artificially constraining its adaptiveness.

2) *Differentiable Detection Framework*: The core challenge in building an adaptive rule-based PIDS is computing the gradients of its parameters ($\partial f / \partial a_n$, $\partial f / \partial g_e$, and $\partial f / \partial thr_e$), as the detection process typically involves a series of discrete, non-differentiable rule-based decisions. To the best of our knowledge, this work is the first to formalize and compute the gradients of parameters in a tag-based PIDS. Our approach achieves this by computing these gradients in response to misclassifications, as detailed in Algorithm 1 (lines 5-6) and Algorithm 2 (lines 5-7). Following the chain rule, the gradient for a parameter such as a_n is given by Equation (5), with similar derivations for other parameters like g_e and thr_e .

$$\frac{\partial \text{Loss}}{\partial a_n} = \sum_{e \in E} \frac{\partial \mathcal{L}(e)}{\partial f} \cdot \frac{\partial f}{\partial a_n} + \underbrace{\alpha \cdot (a_n - a_0)}_{\text{Offline Training}} \quad (5)$$

In CAPTAIN+, we enable gradient-based optimization by modeling the detection process as a differentiable function. We calculate the gradients of each adaptive parameter with respect to the system's loss, allowing us to use gradient descent to find the optimal parameters. The following section details this gradient calculation process, using a node's initial tag value, A , as a representative parameter. The core of our method is to apply

the chain rule recursively through the provenance graph. Since the final detection outcome is a function of node tags, which are themselves functions of prior tags, we can trace the gradients backward from the loss to any initial parameter like A . The recursion begins with the initial state of the graph. The gradient of a tag on a node n' concerning the initial value a_n of a node n is defined as:

$$\frac{\partial tag_{n'}}{\partial a_n} = \begin{cases} 1, & \text{if } n' = n \\ 0, & \text{if } n' \neq n \end{cases} \quad (6)$$

This simply states that a node's tag is initially dependent only on its own starting value.

For the recursive step, we consider the propagation policies detailed in Table III. When an event occurs, an updated tag value, tag_{rule} , is determined by the tags of the source node (tag_{src}) and the destination node (tag_{dest}). By applying the chain rule to these policies, we derive the recursive formula for the gradient. We consolidate the different update rules into the following comprehensive equation:

$$\frac{\partial tag_{n'}}{\partial a_n} = \begin{cases} (1 - g_e) \frac{\partial tag_{dest}}{\partial a_n}, & \text{if } tag_{rule} = c \\ g_e \frac{\partial tag_{src}}{\partial a_n} + (1 - g_e) \frac{\partial tag_{dest}}{\partial a_n}, & \text{if } tag_{rule} = tag_{src} \end{cases} \quad (7)$$

Here, the first case applies when a tag is reset to a constant value c , which halts the flow of the gradient from any prior nodes under this update rule. The second case corresponds to policies where the tag propagates from source to destination. The third case handles policies where the destination's tag remains effectively unchanged (or is influenced only by its prior state).

3) *Online Learning Rate*: The offline training phase benefits from a large volume of events over multiple epochs, allowing for a small, fixed learning rate to achieve gradual and stable convergence. In contrast, the online phase must learn effectively from sparse and critical feedback, necessitating a dynamic learning rate. The strategy, detailed in Algorithm 2 (line 6), differentiates between false positives and false negatives.

For false positives, the learning rate l_{fp} is designed to increase as more errors are observed, allowing the model to correct systematic biases more aggressively over time. It's computed as:

$$l_{fp} = l_{basic} \cdot \frac{N}{n} \quad (8)$$

where l_{fp} is the learning rate, l_{basic} is the initial learning rate, N is the number of false positive events, n is the update interval. The term $\frac{N}{n}$ serves as a scaling factor, increasing the learning rate as more batches of false positives are observed.

False negatives represent critical failures and require a more forceful correction. Therefore, the learning rate for false negatives, l_{fn} , is set to a larger, constant value to ensure that any missed threat prompts a significant model update. Furthermore, because detecting malicious events is the primary goal, the occurrence of a single false negative suggests the model may be too lenient overall. To counteract this, we also adjust the learning rate for future false positives whenever a false negative is found. This adjustment is governed by:

$$l_{fp}^{new} = l_{fp}^{old} \cdot D \quad (9)$$

where D represents the multiplicative factor applied to l_{fp} each time a false negative is detected.

4) *Training and Testing*: In the previous section, we established a differentiable framework for calculating the gradients of our system's adaptive parameters. We now describe how we use gradient descent to optimize these parameters in two distinct phases: an initial offline training phase and a continuous online learning phase during live operation.

The goal of the offline training phase is to learn an optimized set of baseline parameters (A , G , and T) from a static dataset. Instead of random initialization, we begin with a conservative default configuration. This ensures the system maintains high sensitivity to potential threats from the start of the training process. Training proceeds in epochs. In each epoch, the system performs a forward pass over the entire training dataset, propagating tags and accumulating gradients. To ensure stable convergence, we employ a batch update strategy: all gradients are aggregated throughout the epoch, and the model parameters are updated only once at the very end (as shown in Algorithm 1, lines 7-12). The parameter update follows the standard gradient descent rule:

$$p_{new} = p_{old} - l \cdot \frac{\partial Loss}{\partial p_{old}} \quad (10)$$

where p represents any adaptive parameter (e.g., a_n , g_e , or thr_e) and l is the learning rate. This iterative process repeats until a maximum number of epochs is reached or the model's performance converges. The resulting learned parameters form the customized configuration used for initial system deployment.

The online learning phase allows CAPTAIN+ to adapt to evolving threats in real-time, which differs from offline training in several key ways. First, unlike the epoch-based training, online updates are event-driven and immediate. An update is triggered by a misclassification, either a false positive (FP) identified by an analyst or a false negative (FN) discovered later (see Algorithm 2, line 8). Then, the parameter updates are tailored to the error type. FPs trigger updates that make the system more lenient to similar future events, while FNs trigger updates that make it more stringent, thus heightening its sensitivity. Moreover, a misclassification indicates that the tag value of the involved node is unreliable. Therefore, after updating the model parameters, we also immediately update the tag value of the misclassified node itself to prevent cascading errors and improve the accuracy of subsequent, related detections. The effectiveness of this mechanism is verified in §IV-F3. This dual-phase approach ensures that CAPTAIN+ starts with a robust, well-trained model and continues to improve its detection capability through continuous adaptation to real-time feedback.

IV. EVALUATION

The entire system, including data parsing, tag initialization, tag propagation, alarm generation, online learning, and the training/testing framework, comprises approximately 5K lines of Python code. We implement the differentiable tag-propagation framework by maintaining two dictionaries per tag to store gradients for A and G . For T , since its gradients are independent of node tags, there is no need to store them within the nodes.

We deployed CAPTAIN+ and performed all experiments on an Ubuntu 22.04.3 Linux Server with an Intel(R) Xeon(R) Platinum 8358 CPU @ 2.60 GHz and 1.0 TB of memory. Our evaluation aims to answer the following research questions:

- 1) How accurate is CAPTAIN+?
- 2) How efficient is CAPTAIN+?
- 3) How robust is CAPTAIN+ against adversarial attacks?
- 4) How is the long-term performance of CAPTAIN+?
- 5) How do individual modules influence CAPTAIN+'s performance?

A. Experiment Settings

1) *Datasets*: We evaluate CAPTAIN+ using public forensic datasets from the DARPA Transparent Computing program and datasets generated within simulated environments in collaboration with a professional SOC.

- **DARPA TC Datasets** [26]. The DARPA TC program ran between 2016 and 2019 and conducted several red team assessments. It sets up platforms with various background benign activities, generating datasets that include millions of benign and hundreds of malicious events.
- **DARPA OpTC Datasets** [27]. DARPA OpTC program executed a red team simulation of a three-day-long APT attack on a group of hosts, producing data that includes benign activities over seven days and an additional three days with a mix of benign and malicious activities.
- **PKU ASAL Datasets** [28]. The PKU ASAL dataset consists of three hosts: a Linux server, a Windows server, and a Windows 10 host, on which various normal user activities were performed. The dataset also includes five emulated APT attack scenarios, including APT29 and Sidewinder.

2) *Baseline Detectors*: We evaluate CAPTAIN+ against six SOTA PIDS as baselines to compare with CAPTAIN+ from different perspectives. FLASH, KAIROS, and NODLINK are the most recent SOTA embedding-based PIDS and have already evaluated some other embedding-based PIDS [17], [20], [23], demonstrating state-of-the-art performance. These systems are open-source, allowing us to test them on different datasets with their open-sourced code. We chose MORSE since it is an SOTA rule-based PIDS, and this allows us to evaluate the improvements from our differentiable adaptation framework. To ensure a fair and unbiased comparison, we use the settings from MORSE in the evaluation. According to [11], [13] and our communication with the authors, the detection systems of SHADEWATCHER and PROGRAPHER are not fully open-source due to proprietary license restrictions. Although the data preprocessing module of SHADEWATCHER is open-source, we were unable to locate the preprocessing output files used for training and testing. Therefore, for these systems, we utilized the available code to evaluate the efficiency and latency of data preprocessing, and relied on the detection results reported in their paper to assess detection accuracy. We reimplemented MORSE according to their paper. All code used for evaluating the baselines will also be publicly available for future research.

TABLE V
DATASET STATISTICS FOR TRAINING AND TESTING

Dataset	Training		Testing	
	Duration	Size	Duration	Size
T-3	174:55	15.7M	88:10	5.2M
C-3	77:53	1.9M	185:36	5.4M
T-5	70:45	119.6M	177:38	312.0M
C-5	71:15	59.8M	177:50	208.8M
OpTC	87:21	47.2M	18:25	3.73M
S-1	5:23	8.5M	1:28	3.6M
S-2	5:01	1.2M	1:31	629K
S-3	5:13	209.0K	1:29	113K

3) *Experiment Setup: Initial State and Data Splits:* Following prior work [19], we assume that attacks originate from external networks or existing files. Consistent with the conservative strategy mentioned in §III-A, we set their initial integrity score in A0 to 0, which signifies low integrity. To ensure a clean training process, we partition our datasets based on the ground truth attack timestamps. All data preceding the first attack is used as the training set (containing only benign events), and the remainder is used for testing. The specific details for each dataset split are provided in Table V.

Cross-Validation: To prevent overfitting and respect the temporal nature of audit logs, we employ time-series cross-validation instead of standard k -fold cross-validation. Standard k -fold shuffling would violate temporal causality, potentially training the model to predict past events based on future ones. Our approach involves setting a time window (approximately 2/3 the length of the training set) and creating k distinct training/validation splits by sliding this window forward in time. For our experiments, we set $k = 3$. This method provides a robust and unbiased estimate of the model’s performance on unseen streaming data.

Hyperparameter Selection: The performance of CAPTAIN+ depends on several hyperparameters, particularly the three regularizer coefficients (α, γ, τ). We propose a two-pronged strategy for their selection. When a labeled validation set is available, we use grid search to identify the optimal hyperparameter combination. Our optimization goal is to select settings that yield the fewest false positives while maintaining or improving the true positive rate compared to a conservative baseline. For example, for the E3-CADETS dataset, this process resulted in optimal settings of $\alpha = 0.1, \gamma = 0.1, \tau = 0.1$ with a learning rate of 0.01.

In practice, comprehensive validation sets for PIDS are often unavailable. For these scenarios, we propose a heuristic-based method to estimate the coefficients. Since our training begins from a conservative state, false alarms serve to “loosen” the rules. The regularizers counteract this, preventing the model from becoming too lenient and missing true attacks. The magnitude of the coefficients, therefore, should depend on the number of false alarms (N) an administrator is willing to tolerate per entity. Based on this principle, we provide the following heuristic to estimate the coefficients before training: $\alpha \approx 3N, \gamma \approx 3N, \tau \approx 12N$.

Moreover, we provide a mathematical justification for this heuristic. According to Eq. 5, setting $\frac{\partial \text{Loss}}{\partial a_n}, \frac{\partial \text{Loss}}{\partial g_e}$ and $\frac{\partial \text{Loss}}{\partial thr_e}$ be

zero yields the following conditions:

$$\begin{aligned}\alpha &= - \sum_{e \in E} \frac{\partial \mathcal{L}(e)}{\partial f} \cdot \frac{\partial f}{\partial a_n} \cdot (a_n - a_0)^{-1} \\ \gamma &= - \sum_{e \in E} \frac{\partial \mathcal{L}(e)}{\partial f} \cdot \frac{\partial f}{\partial g_e} \cdot (g_e - g_0)^{-1} \\ \tau &= - \sum_{e \in E} \frac{\partial \mathcal{L}(e)}{\partial f} \cdot \frac{\partial f}{\partial thr_e} \cdot (thr_e - thr_0)^{-1}\end{aligned}\quad (11)$$

Since the regularization coefficients α, γ and τ are used in offline learning, the loss function $L(e) = \max(0, (1 - f(e))^2 - 1)$. For a misclassified event where $-1 < f(e) < 0$, the derivative is therefore:

$$- \sum_{e \in E} \frac{\partial \mathcal{L}(e)}{\partial f} = \sum_{e \in E} 2(1 - f(e)) \quad (12)$$

$$\alpha = \sum_{e \in E} 2(1 - f(e)) \cdot \frac{\partial f}{\partial a_n} \cdot (a_n - a_0)^{-1} \approx 3N(n)$$

$$\gamma = \sum_{e \in E} 2(1 - f(e)) \cdot \frac{\partial f}{\partial g_e} \cdot (g_e - g_0)^{-1} \approx 3N(e)$$

$$\tau = \sum_{e \in E} 2(1 - f(e)) \cdot \frac{\partial f}{\partial thr_e} \cdot (thr_e - thr_0)^{-1} \approx 12N(e) \quad (13)$$

At the optimal point where the gradients of the total loss are zero, we also have the following relationships for the partial derivatives: $\frac{\partial f}{\partial thr_e} = -1, -1 < \frac{\partial f}{\partial a_n} < 0, -1 < \frac{\partial f}{\partial g_e} < 0$. According to §III-A, the default values are typically $a_0 = 0$ and $g_0 = 1$. We can make the following estimations: $f(e) \approx -0.5, \frac{\partial f}{\partial a_n} \approx 0.5, a_n - a_0 \approx 0.5, \frac{\partial f}{\partial g_e} \approx -0.5, g_e - g_0 \approx -0.5, thr_e - thr_0 \approx -0.25$. Let $N(n)$ and $N(e)$ be the number of allowed false alarms related to node n and event e .

It is important to note that the values above are heuristic estimates. Since attackers may hide within training data or exploit benign entities, the choice of regularizer coefficients depends on the presumed purity of the “benign” dataset. Therefore, a universally optimal method for selecting these coefficients does not exist. Instead, we propose a dynamic adjustment strategy that begins with the estimated values and adapts based on model performance. If missed alarms appear when compared to the conservative baseline, the regularization is made stricter by multiplying the coefficients by a set factor (e.g., 10). Conversely, after a prolonged period with no missed alarms, the regularization can be relaxed by dividing the coefficients by the same factor, after which the model is retrained.

Online Learning Trigger: The online learning mechanism is triggered whenever a misclassification is observed. In our experiments, we simulated this by comparing the system’s output against a labeled ground truth dataset containing an average of 70 malicious events each. In a real-world deployment, this trigger would be the feedback from a security analyst who either dismisses a false alarm or identifies a missed attack.

TABLE VI
COMPARISON OF EVENT-LEVEL ACCURACY

	T-3	C-3	TH-3	OpTC	S-1	S-2	S-3
Total True Alarms							
MORSE	22	22	7	18	7	5	10
CAPTAIN	22	22	7	15	7	5	10
CAPTAIN+	27	22	7	20	8	5	15
Total False Alarms							
MORSE	52.1K	13.4K	490.4K	25.5K	3.3K	203	60
CAPTAIN	3.1K	311	100.9K	20.0K	94	69	38
CAPTAIN+	416	226	447	6.8K	22	11	13

4) *Data Labeling*: We labelled entities and events in each attack scenario’s kill chain as malicious and all others as benign, based on the corresponding attack report. While prior studies [11], [17] offer entity-level labels in their open-source repositories, they often lack a specified labeling strategy or sufficient contextual details. Furthermore, we observed an impractically high number of malicious labels in their datasets. For instance, over 12,000 system entities were marked as malicious within 30 hours in the CADETS dataset from Engagement 3, a figure that does not reflect a realistic SOC environment. To ensure a fair and unbiased comparison between CAPTAIN+ and baseline systems, we created and utilized our own data labels for this evaluation. This labeling process was non-trivial, as it required mapping descriptions from natural language attack reports to records in raw audit logs. To facilitate future research and improve reproducibility, we have made our data labels publicly available.

B. How accurate is CAPTAIN+?

1) *False Alarm Events Reduction*: We first compared the number of false alarms generated by CAPTAIN+, CAPTAIN, and MORSE across different datasets. As shown in Table VI, CAPTAIN+ reduces false alarms by over 76% on average compared with CAPTAIN. For those remaining false alarms, especially in S-2 and S-3, CAPTAIN+ cannot remove them for the following reasons. The persistence of some false alarms, particularly in datasets S-2 and S-3, can be attributed to several factors. First, some “false” alarms relate to potentially malicious nodes (e.g., specific configuration files in S-2) but cannot be directly mapped to attack steps and are therefore not labeled as malicious. Second, some relevant nodes and edges are absent from the training set or fail to trigger alarms.

Furthermore, since CAPTAIN+ adopts the same initial training method as CAPTAIN, their parameter settings are identical at the start of the detection phase, making some initial false positives unavoidable before online learning is triggered. Finally, to ensure learning stability, we employ a gradual increase in the learning rate, meaning that some initial parameter updates do not have an immediate, significant effect. To verify that this reduction in false positives does not compromise detection capability, we also analyzed the number of malicious events detected. Table VI shows that CAPTAIN+ detects all malicious events that CAPTAIN does, and in some datasets (e.g., T-3, OpTC), detects even more.

TABLE VII
COMPARISON OF NODE-LEVEL ACCURACY

	TP	FP (0-hop)	FP (1-hop)	FN (0-hop)
Engagement 3 CADETS				
FLASH	16	4503	4485	10
KAIROS	15	1017	1003	11
NODLINK ¹	3	120	114	2
MORSE	16	51	43	10
CAPTAIN	16	34	26	10
CAPTAIN+	16	31	23	10
Engagement 3 TRACE				
FLASH	5	27202	27178	19
NODLINK ¹	4	170	170	0
MORSE	10	243	234	14
CAPTAIN	10	12	11	14
CAPTAIN+	12	7	6	12
Engagement 3 THEIA				
FLASH	2	53230	53050	13
KAIROS	12	3566	3422	3
NODLINK ¹	4	62	58	0
MORSE	11	220	213	4
CAPTAIN	11	194	187	4
CAPTAIN+	11	83	81	4
OpTC Dataset				
FLASH	19	81690	79859	2
KAIROS	20	39505	38373	1
MORSE	17	314	306	4
CAPTAIN	15	307	299	6
CAPTAIN+	19	286	178	2

¹NODLINK only provides detected processes and is evaluated on its accuracy.

2) *False Alarm Entity Reduction*: Although CAPTAIN+ is an event-level detector, we also compare it with state-of-the-art (SOTA) entity-level PIDS, such as FLASH, KAIROS, and NODLINK. To make this comparison feasible, we transformed the alarm events triggered by MORSE, CAPTAIN, and CAPTAIN+ into alarm entities, where an entity is considered an alarm entity if it is involved in any alarm event. To ensure the integrity of our comparison, we also addressed differences in dataset usage and evaluation metrics. While previous works [11], [13], [17] often used only a subset of each DARPA dataset, we utilized the entire dataset to simulate a more realistic SOC scenario. To do so, we divided the data as described in §IV-A3, retraining baseline models using their provided code. Furthermore, we observed that some baselines, such as FLASH and THREATTRACE, employed lenient criteria for counting false positives (FPs) [13]. To ensure a fair yet strict evaluation, we report results that include a ‘one-hop FP’ metric, where we exclude any FP entities within a one-hop distance of a ground-truth attack entity. These differences in experimental settings account for potential discrepancies between our results and those in the referenced papers.

We evaluated detection accuracy on the CADETS, TRACE, THEIA, and OpTC datasets. As shown in Table VII, the performance of the baseline systems varied significantly. Among the embedding-based systems, FLASH reported a fair number of true positives (TPs). Still, it produced an excessive number of FPs, as its GNN-based approach struggles with limited training data. KAIROS performed well in time-window detection but exhibited poor entity-level accuracy. Similarly, NODLINK detected most attacks but produced many FPs and missed key entities such as files and network sockets. The rule-based system, MORSE,

achieved a lower false positive rate at the node level. This is attributable to its semantically-rich alarms, which provide additional context for filtering. For example, during a file corruption event, MORSE flags both the process and the corrupted file, but the file node may be filtered from the final alert because it is considered an object of the attack rather than a malicious actor itself. This explains why MORSE can trigger thousands of event-level false alarms but only flags a few hundred nodes incorrectly. Such semantic filtering is not available in embedding-based PIDS.

On all datasets, CAPTAIN+ demonstrates superior performance, identifying the majority of TPs while maintaining fewer FPs, even when compared to CAPTAIN. Since CAPTAIN+ operates at the event level, some entities not directly associated with attack events may not be reported. For instance, when malware is downloaded and executed, CAPTAIN+ triggers MalFileCreation and FileExec alarms, providing analysts with critical information about the malware process, file, and parent process. However, to prevent alarm fatigue, we do not immediately report the entry network node used by the attackers. This design choice leads to some false negatives (FNs) in CAPTAIN+, with the expectation that analysts can investigate these related nodes after the initial response.

C. How efficient is CAPTAIN+?

In this section, we compare CAPTAIN+ with the SOTA PIDS, including FLASH, KAIROS, NODLINK, SHADEWATCHER, MORSE and CAPTAIN, to evaluate their efficiency.

1) *Detection Latency*: In our evaluation, we define detection latency as the sum of three components: buffer time (delays from time-windowing, as discussed in §II), preprocessing time (e.g., data parsing, feature extraction), and detection time (from data input to result output). Our latency evaluation focuses on the testing phase, as training can be conducted offline, and utilizes the TRACE and CADETS datasets. To ensure a fair comparison, we executed the baseline systems' original code, excluded I/O time to measure only computational latency, and verified that the amount of data fed into each detection module after preprocessing was identical.

The results in Table VIII show that CAPTAIN+ outperforms other PIDS in terms of both preprocessing and detection time. Unlike other methods that rely on embedding graph structures and require significant buffer time (from minutes to hours), CAPTAIN+ processes audit logs in a streaming fashion, eliminating buffer time entirely, and uses a lightweight preprocessing pipeline, enabling faster data handling.

Besides, compared to other rule-based systems like MORSE and CAPTAIN, the introduction of the online learning mechanism in CAPTAIN+ means that every false positive or false negative triggers a parameter adjustment. As a result, CAPTAIN+ consumes more time during the detection stage. However, it is essential to note that the detection logic in CAPTAIN+ remains unchanged, so this additional time should be considered part of the training time. The extra time spent is, on average, less than one minute, which we consider a negligible cost for improved accuracy.

TABLE VIII
COMPARISON OF DETECTION LATENCY

	Buffer Time	Preprocessing Time	Detection Time
Engagement3 TRACE			
Flash	57:49	107:50	64:24
ShadeWatcher	N/A ¹	100:22	3:40 ²
NodLink	00:10	135:42	2:48
MORSE	0	58:20	1:29
CAPTAIN	0	58:20	1:31
CAPTAIN+	0	58:20	2:29
Engagement3 CADETS			
Kairos	15:00	15:34	29:46
Flash	82:52	18:57	7:41
NodLink	00:10	6:18	6:41
MORSE	0	7:22	1:19
CAPTAIN	0	7:22	1:23
CAPTAIN+	0	7:22	2:03

¹We did not find a clear number in their codes or paper.

²SHADEWATCHER extracts the last 10% interactions as the testing set, while the testing set of us is around 2.5 times larger.

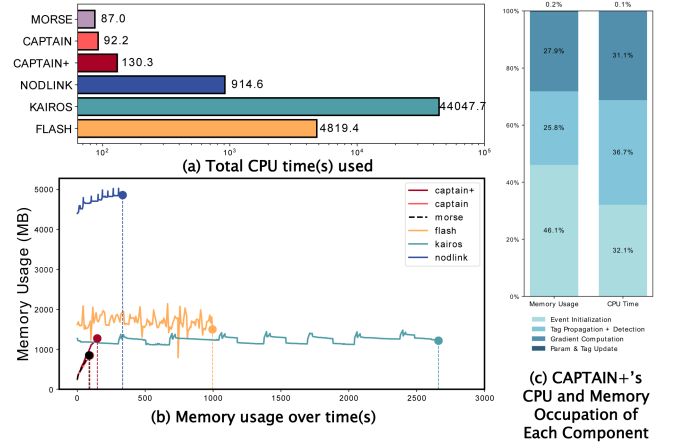


Fig. 3. Comparison of Resource Consumption

2) *Runtime Overhead*: Another key metric for PIDS evaluation is runtime overhead. This section assesses CAPTAIN+'s runtime overhead and memory usage during detection. We use the Python "resource" module to measure the resource consumption of detectors, running them in CPU mode. As GPUs may not be available in most real-world threat detection scenarios. We calculate the total CPU time in user mode to evaluate CPU resource consumption, as shown in Fig. 3(a). Due to the use of deep learning libraries, KAIROS, FLASH and NODLINK are multi-threaded, so their total CPU time in Fig. 3(a) may exceed the detection time reported in Table VIII. In contrast, MORSE, CAPTAIN, and CAPTAIN+ are single-threaded. For fair comparison, the CPU time sum of all cores is considered when conducting multi-core execution statistics. Since CAPTAIN+ relies on basic arithmetic operations on real-number tags, it requires far less CPU time than embedding-based PIDS (around 3% of FLASH and 0.3% of KAIROS), which rely on complex matrix and vector operations for their neural networks. Moreover, like the detection latency experiment, there may be a slight increase in CPU usage compared to CAPTAIN and MORSE.

We used the “psutil” library in Python to monitor the memory usage of the detectors during detection. As shown in Fig. 3(b), CAPTAIN+ completes detection much faster than embedding-based PIDS (as shown in Table VIII) and uses the least memory throughout the entire detection process. As seen in Fig. 3(b), compared to CAPTAIN and MORSE, CAPTAIN+ shows a slight increase in both memory usage and detection time. This is partly due to adjustments made for each false alarm, leading to multiple calls. However, this memory is only used for online learning and does not affect actual detection. In practice, users batch and periodically update false alarms, further reducing memory use. When focusing only on detection, comparing with CAPTAIN offers a clearer view. The small difference in memory usage between CAPTAIN+ and MORSE comes from the storage of fine-grained rules. Overall, CAPTAIN+ retains the lightweight and fast characteristics of rule-based PIDS.

To further clarify CAPTAIN+’s runtime overhead, we present a resource consumption proportion in Fig. 3(c). As the figure illustrates, the overhead is primarily due to the online learning module (specifically, gradient calculation), which accounts for approximately 30% of the additional cost. We believe this is a reasonable trade-off for achieving real-time model adaptation compared to the significantly higher cost of periodic full model retraining.

D. How robust is CAPTAIN+ against adversarial attacks?

As the development of PIDS grows, adversarial attacks aimed at evading these detection mechanisms are a growing concern. This section examines the robustness of CAPTAIN+ against “mimicry attacks” and “data poisoning attacks”.

1) *Adversarial Mimicry Attack*: Mimicry attacks on PIDS involve altering the provenance data and incorporating more benign features to “mimic benign behavior”, to evade detection. We evaluated the robustness of CAPTAIN+ against mimicry attacks using the attack methodology from [11], [33], i.e., inserting benign structures into the attack graphs. Our mimicry attack involves two steps: First, we extracted common benign event structures from the training data. Then, we swapped the original entities in these benign structures with malicious ones from an actual attack graph. This creates a malicious action that is disguised to look like everyday system activity. For our experiment, we used the *CADETS* dataset (Engagement 3) due to its manageable size and focused on testing the system’s response to the specific event (XIM, chmod, /tmp/test) after it was cloaked in a benign structure.

As Fig. 4(a) shows, both CAPTAIN+ and FLASH demonstrated some robustness against the mimicry attack. The attack entity (/tmp/test) was still detected by both systems even after adding benign activities. However, we observed a notable drop in FLASH’s anomalous score with just a few added benign events (fewer than 10), a result also confirmed in [11]. On the other hand, CAPTAIN+ remained unaffected by the mimicry attack, showing better robustness than the baseline.

CAPTAIN+ achieves robustness through three key factors: 1) Heuristic-based propagation: Unlike embedding-based PIDS like FLASH, CAPTAIN+ uses heuristic rules that are less affected

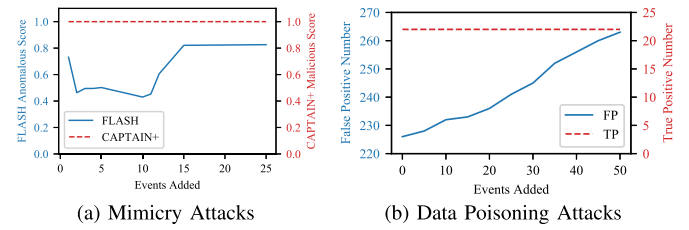


Fig. 4. Impact of Adversarial Attacks on CAPTAIN+

by minor benign changes. 2) Event-level detection: CAPTAIN+ works at a finer granularity (event level) compared to entity-level detectors. Since operating at event level without buffering, CAPTAIN+ remains unaffected by benign events added after attacks—unlike time-window-based detectors. 3) Conservative, rule-based detection: CAPTAIN+ is not anomaly-based; it uses conservative rules trained on normal data to reduce false alarms. Therefore, adding normal features does not weaken its detection capability, as the conservative rules ensure detection reliability.

2) *Data Poisoning Attack*: Data poisoning refers to injecting malicious events into benign training data, causing the detector to learn attacker-crafted patterns and potentially miss future attacks. To enhance CAPTAIN+’s robustness, we add a regularizer to the loss function to reduce sensitivity to training data and mitigate the impact of such manipulation.

Training Set Poisoning Attack: In this section, we demonstrate CAPTAIN+’s resilience against training set poisoning attack using a real attack scenario, namely, a Pine Backdoor and Phishing Email attack from the Engagement 3 TRACE dataset. In this attack, a vulnerable pine application connected to the email server 128.55.12.73, downloaded and executed a phishing email attachment. However, since 128.55.12.73 had multiple activities, it caused some “false” alarms in the training data. This violates our assumption (as noted in §II-D) that the training set should be free from attack entities or events, making it an ideal test case for dataset poisoning.

To analyze the attack, we focus on the initial integrity tag (a) of the IP node 128.55.12.73, which shows the impact of the regularization term. Without the regularizer, 128.55.12.73 would receive a high integrity score (close to 1.0) due to its benign activities in the training data, which would cause us to miss the phishing email attack originating from it. However, when we apply the regularizer ($\alpha = 10$), CAPTAIN+ becomes more cautious during training, assigning a lower integrity score of 0.488 to the IP, similar to the “grey node” example in §II-E. This helps prevent false alarms and ensures that phishing email attacks are correctly detected. More details can be found in §IV-G. Additionally, the event propagation rate for the attack (pine, read, 128.55.12.73) becomes more conservative, helping to mitigate the effects of the poisoned training set.

Online Learning Poisoning Attack: To evaluate the robustness of CAPTAIN+ against online learning poisoning attacks, we simulated an adversary attempting to influence the system by injecting misleading benign events. The experiment was conducted on the CADETS dataset from Engagement 3, focusing on the malicious event (XIM, chmod, /tmp/test). The attack

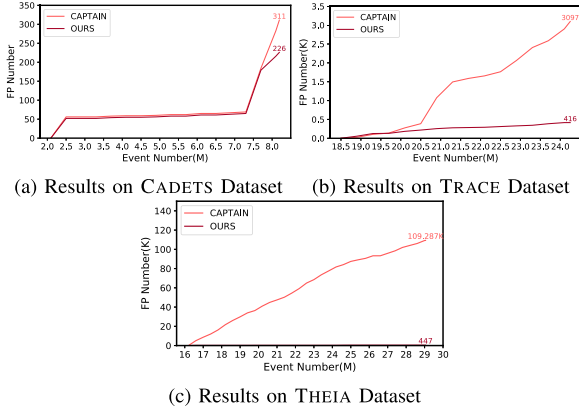


Fig. 5. Comparative Analysis of False Positives in Long Term

was simulated by creating a malicious file, `/tmp/test`, that mimicked the interaction patterns of a benign file (`/dev/tty`) and by inserting preceding executions by benign processes (e.g., `newsyslog`) as noise. As shown in Fig. 4(b), while the injection of these false events led to a gradual increase in false positives, CAPTAIN+ consistently maintained its detection of the true positive event. This robustness is primarily attributable to the controlled online learning rate, which prevents the model from overreacting to misleading feedback and incorrectly reclassifying a malicious event as benign. Overall, CAPTAIN+ proved highly robust against this adversarial attack.

E. How is the long-term performance of CAPTAIN+?

In this section, we compare the adaptability of CAPTAIN+ and CAPTAIN to concept drift by tracking how their false positives change over time. We use the detected events number as a proxy for time and evaluate both systems on the CADETS, TRACE, and THEIA datasets from DARPA Engagement 3.

As shown in Fig. 5, CAPTAIN+ outperforms CAPTAIN in controlling false positives, both in the final results and in the rate of increase over time. While CAPTAIN can manage false positives well in the early stages, its rate of false positives increases rapidly as time goes on. In contrast, CAPTAIN+ maintains a relatively low false positive rate throughout the detection process, particularly in the TRACE and THEIA datasets. In the CADETS dataset, the false positive rate is already low, so the effect of online learning is less noticeable. The key to CAPTAIN+’s performance lies in its online learning mechanism. After each wrong detection occurs, the system updates its parameters and node tags, enabling CAPTAIN+ to control false positives and adapt to new patterns effectively.

Online learning not only reduces false positives but also addresses missed attacks. For example, when the event “(ztmp, write, 162.66.239.75)” first appeared in the TRACE dataset, both CAPTAIN+ and CAPTAIN missed it. However, when the event occurred again, CAPTAIN+ flagged it correctly after online learning, whereas CAPTAIN missed it again.

F. How do modules influence CAPTAIN+’s performance?

1) *Ablation Study on Learning Rate:* During training, the learning rate not only determines the learning speed but also

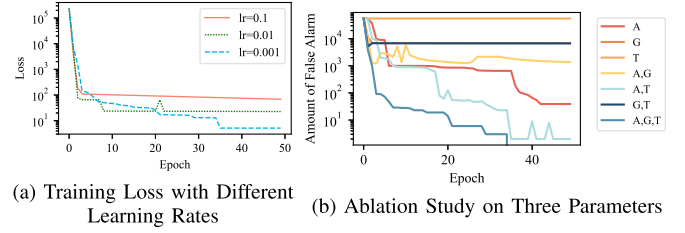


Fig. 6. Influence of Parameters on Detection Performance

controls the granularity of the search. We evaluated the impact of the learning rate using the CADETS dataset. As shown in Fig. 6(a), a higher learning rate leads to a sharper decline in training loss during the initial epochs. In contrast, a lower learning rate enables finer optimization, yielding a lower final loss when sufficient epochs are available. This necessitates a balance between training speed and final model quality. Based on the figure, if training is limited to 20 epochs due to resource constraints, a learning rate of 0.01 is preferable. For longer training durations, however, a rate of 0.001 achieves superior results.

2) *Ablation Study on Three Parameters:* We conducted an ablation study to evaluate the contribution of each adaptive parameter (A , G , and T). Since the parameters can be adjusted independently, we performed seven experiments, each optimizing a different subset of the parameters: $\{A\}$, $\{G\}$, $\{T\}$, $\{A, G\}$, $\{A, T\}$, $\{G, T\}$, and $\{A, T, G\}$. The results, presented in Fig. 6(b), reveal several key findings. First, adjusting the alarm threshold (T) alone has a limited impact on performance. This is because, without changes to the initial tags or propagation rates, propagated tag values tend to polarize towards 0 or 1, rendering a continuous threshold ineffective. The results also indicate that tuning the tag initialization parameter (A) is the most effective single-parameter strategy for reducing false alarms, playing a more critical role than G or T .

While the combination of all three parameters (A , G , and T) generally yields incremental improvements, we observed that adjusting T alone sometimes resulted in fewer false alarms on specific datasets (S-3 and C-3). This occurs because, without the ability to modulate the propagation rate or threshold, the optimization process compensates by assigning higher integrity tags to a broader set of nodes. Although this can reduce false alarms in a coarse-grained manner, it carries the risk of making the system overly lenient. Finally, a significant advantage of tuning all three parameters is the accelerated convergence of the training process. As shown in Fig. 6(b), while the $\{A\}$, $\{A, T\}$, and $\{A, T, G\}$ configurations achieve comparable final detection performance on the testing set, the full $\{A, T, G\}$ model converges in considerably fewer epochs during training.

3) *Ablation Study on Online Learning Module:* To assess the contribution of each component within the online learning module, we conducted an ablation study on three key submodules: “False Negative Feedback,” “Node Tag Update,” and “Learning Rate Adjustment,” with the results presented in Table IX. Disabling the False Negative Feedback loop resulted in a further reduction in false positives, but at the unacceptable

TABLE IX
ABLATION STUDY ON ONLINE LEARNING MODULE

	# of TP	# of FP
w/o false negative feedback	26	213
w/o learning rate adjustment	49	1.1K
w/o node tag update	44	606
with learning rate set to 0	49	3.4K
CAPTAIN (w/o all modules)	44	3.4K
CAPTAIN+ (with all modules)	49	642
Ground-truth	60	0

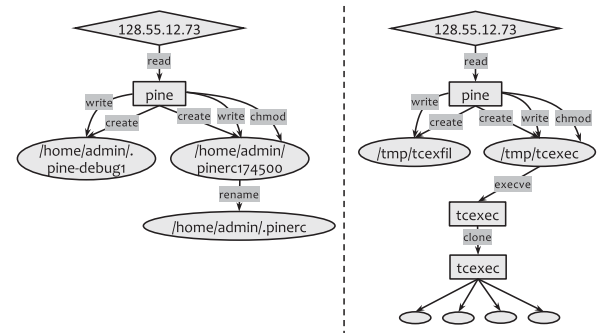
cost of missing attacks due to excessive leniency. Similarly, removing the Learning Rate Adjustment mechanism and using a fixed learning rate resulted in an increase in false positives, indicating that dynamic adjustment is crucial for correcting false positives. Finally, removing the Node Tag Update submodule also reduced false positives but significantly limited CAPTAIN+'s responsiveness to and ability to correct for false negatives. Collectively, these experiments demonstrate that each submodule is critical for achieving the desired balance between high detection accuracy and a low false positive rate.

G. Case Study

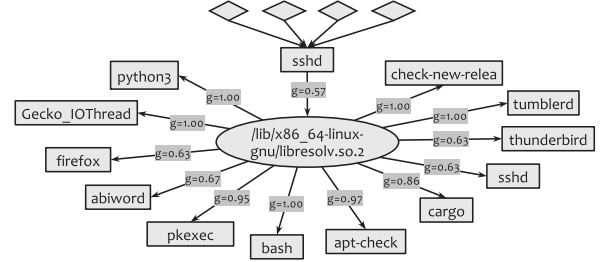
Another advantage of rule-based PIDS over embedding-based PIDS is the interpretability of the detection process. To investigate the explainability of CAPTAIN+, we delve into the training outcomes of the E3-TRACE dataset as a case study. The case study highlights CAPTAIN+'s key strength—fine-grained detection enabled by adaptive parameters. As mentioned in §II-E and §III, unlike prior work [18], [19], [20] that uses discrete, manually assigned values and global rules, CAPTAIN+ tunes customized numerical parameters for each entity and event, improving its ability to distinguish subtle differences between benign and malicious behavior.

Fig. 7(a) demonstrates how CAPTAIN+ better distinguishes similar behaviors using fine-tuned parameters. In both cases, process `pine` reads from `128.55.12.73` (possibly the email server) and creates, writes, and changes the permissions of a new file. Traditional rule-based PIDS like MORSE [18], [19] struggle to assign a tag for `128.55.12.73`: marking the IP as benign (tag = 1.0) misses the first stage of attacks, while assigning the initial tag as “Untrusted” causes multiple false “Malicious File Creation” and “Change Permissions” alarms on the left-hand side benign graph. Instead, CAPTAIN+ assigns a moderate initial tag a_n , 0.488, to this IP and slightly tunes down the threshold thr for the event (`pine`, `create`, `/home/admin/pinerc174500`) from 0.5 to 0.478. As $0.478 < 0.488$, the creation of the file will not be alarmed, but in real attacks, the thr for the event (`pine`, `create`, `/tmp/tcexec`) is still 0.5, meaning creating a malicious file would still trigger alarms ($0.5 > 0.488$).

Unlike MORSE's fixed decay factors, CAPTAIN+ more precisely addresses the dependency explosion caused by `libresolv.so.2` through fine-grained propagation rates, learning distinct values for different events. As shown in Fig. 7(b), CAPTAIN+ learns to assign lower propagation rates to events



(a) The graph on the left side represents normal behavior in the training set, while the graph on the right side depicts an attacker executing a malicious email attachment.



(b) CAPTAIN+ assigns different propagation rates (g_e).

Fig. 7. Case study from Engagement 3 TRACE. We omitted some irrelevant details to keep the graphs clean and succinct.

that frequently lead to dependency explosions (e.g., when `ssh` or `firefox` reads the file), while even for the same process, `firefox`, CAPTAIN+ learns to suppress the dependency propagation when reading this file ($g_e = 0.63$) and remains sensitive to maliciousness propagation when loading this file ($g_e = 0.99$). This adaptive balance prevents excessive leniency while ensuring no malicious activities are missed.

With its enhanced capability of distinguishing, CAPTAIN+ can significantly reduce false alarms compared to traditional rule-based PIDS while ensuring that all true attacks are detected. These fine-grained parameters of CAPTAIN+ outperform many embedding-based PIDS in detecting mimicry attacks. For instance, in Fig. 7(a), the attacker can modify the file name of `/tmp/tcexec` for a mimicry attack with the knowledge that `pine` might interact with a file like `/home/admin/pinerc+numbers`. Since generalization techniques like Word2Vec and Sentence2Vec are widely used in embedding-based PIDS, they can manipulate the file name by appending random digits to `pinerc`, making it similar to `pinerc174500` after embedding. However, the attacker requires the exact file name of `pinerc174500` to evade CAPTAIN+'s detection, making mimicry much more complicated, especially with limited training data (only four related events in 7 days).

V. DISCUSSION AND CONCLUSION

A. More Optimization Algorithms

For parameter optimization, we utilize Batch Gradient Descent (BGD). The choice of BGD over mini-batch or stochastic

methods is deliberate. In our context, a “batch” encompasses all events within a single analysis window. Updating the adaptive parameters before processing the entire graph, as would occur with smaller batches, could introduce tag inconsistencies during the propagation process. Therefore, a full-batch update is necessary to maintain analytical integrity. Our differentiable tag propagation framework, however, is not limited to BGD. Its ability to provide first-order gradients makes it directly compatible with numerous other optimizers, such as Momentum, NAG [50], ADADELTA [51], ADAM [37], and RMSPROP, which often achieve faster and more stable convergence. We thus leave the implementation of these alternative optimization strategies for future investigation.

B. Conclusion

This paper introduces CAPTAIN+, a tag-based Provenance Intrusion Detection System (PIDS) that automatically adapts to diverse environments. Its adaptability stems from three tunable parameters: tag initialization (A), propagation rate (G), and detection threshold (T). We propose a differentiable tag propagation framework that enables the optimization of these parameters via gradient descent. To our knowledge, CAPTAIN+ is the first PIDS to leverage gradient-based methods for automated parameter tuning, overcoming the rigidity of traditional rule-based systems. During detection, CAPTAIN+ employs online learning to refine its configuration in real-time continually. By learning from both false positives and false negatives, it ensures sustained accuracy — a capability absent in statically trained offline models.

REFERENCES

- [1] A. Alshamrani, S. Myneni, A. Chowdhary, and D. Huang, “A survey on advanced persistent threats: Techniques, solutions, challenges, and research opportunities,” *IEEE Commun. Surveys Tuts.*, vol. 21, no. 2, pp. 1851–1877, Secondquarter 2019.
- [2] H. B. Review, “Missed alarms and 40 million stolen credit card numbers: How target blew it,” (n.d.). [Online]. Available: <https://hbr.org/2014/03/could-target-have-prevented-its-security-breach>
- [3] N. Y. Times, “Equifax says cyberattack may have affected 143 million the U.S.,” (n.d.). [Online]. Available: <https://www.nytimes.com/2017/09/07/business/equifax-cyberattack.html>
- [4] G. P. Spathoulas and S. K. Katsikas, “Using a fuzzy inference system to reduce false positives in intrusion detection,” in *Proc. 2009 16th Int. Conf. Syst., Signals Image Process.*, 2009, pp. 1–4.
- [5] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM Comput. Surv.*, vol. 41, no. 3, pp. 1–58, 2009, doi: [10.1145/1541880.1541882](https://doi.org/10.1145/1541880.1541882).
- [6] C. Dietrich, K. Krombholz, K. Borgolte, and T. Fiebig, “Investigating system operators’ perspective on security misconfigurations,” in *Proc. 2018 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 1272–1289, doi: [10.1145/3243734.3243794](https://doi.org/10.1145/3243734.3243794).
- [7] S. C. Sundaramurthy et al., “A human capital model for mitigating security analyst burnout,” in *Proc. 11th Symp. Usable Privacy Secur.*, 2015, pp. 347–359.
- [8] B. A. Alahmadi, L. Axon, and I. Martinovic, “99% false positives: A qualitative study of SOC analysts’ perspectives on security alarms,” in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 2783–2800.
- [9] S. C. Committee, “A “kill chain” analysis of the 2013 target data breach,” 2014. [Online]. Available: <https://www.commerce.senate.gov/services/files/24d3c229-4f2f-405d-b8db-a3a67f183883>
- [10] F. Dong et al., “Are we there yet? An industrial viewpoint on provenance-based endpoint detection and response tools,” in *Proc. 2023 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2023, pp. 2396–2410.
- [11] M. U. Rehman, H. Ahmadi, and W. U. Hassan, “Flash: A comprehensive approach to intrusion detection via provenance graph representation learning,” in *Proc. 2024 IEEE Symp. Secur. Privacy*, 2024, pp. 139–139.
- [12] F. Yang, J. Xu, C. Xiong, Z. Li, and K. Zhang, “ProGrapher: An anomaly detection system based on provenance graph embedding,” in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 4355–4372.
- [13] Z. Cheng et al., “Kairos: Practical intrusion detection and investigation using whole-system provenance,” 2023, *arXiv:2308.05034*.
- [14] A. Goyal, G. Wang, and A. Bates, “R-Caid: Embedding root cause analysis within provenance-based intrusion detection,” in *Proc. 2024 IEEE Symp. Secur. Privacy*, 2024, pp. 257–257.
- [15] J. Zengy et al., “ShadeWatcher: Recommendation-guided cyber threat analysis using system audit records,” in *Proc. 2022 IEEE Symp. Secur. Privacy*, 2022, pp. 489–506.
- [16] S. Li et al., “NodLink: An online system for fine-grained APT attack detection and investigation,” 2023, *arXiv:2311.02331*.
- [17] S. Wang et al., “THREATTRACE: Detecting and tracing host-based threats in node level through provenance graph learning,” *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 3972–3987, 2022.
- [18] M. N. Hossain et al., “SLEUTH: Real-time attack scenario reconstruction from COTS audit data,” in *Proc. 26th USENIX Secur. Symp.*, 2017, pp. 487–504.
- [19] M. N. Hossain, S. Sheikhi, and R. Sekar, “Combating dependence explosion in forensic analysis using alternative tag propagation semantics,” in *Proc. IEEE Symp. Secur. Privacy*, 2020, pp. 1139–1155.
- [20] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, and V. Venkatakrishnan, “Holmes: Real-time APT detection through correlation of suspicious information flows,” in *Proc. IEEE Symp. Secur. Privacy*, 2019, pp. 1137–1152.
- [21] C. Xiong et al., “Conan: A practical real-time apt detection system with high accuracy and efficiency,” *IEEE Trans. Dependable Secur. Comput.*, vol. 19, no. 1, pp. 551–565, Jan./Feb. 2022.
- [22] W. U. Hassan et al., “NoDoze: Combatting threat alert fatigue with automated provenance triage,” in *Proc. 26th Annu. Netw. Distrib. Syst. Secur. Symp.*, 2019.
- [23] X. Han, T. F. J. Pasquier, A. Bates, J. Mickens, and M. I. Seltzer, “Unicorn: Runtime provenance-based detector for advanced persistent threats,” in *Proc. 27th Annu. Netw. Distrib. Syst. Secur. Symp.*, 2020.
- [24] W. Lingzhi et al., “Incorporating gradients to rules: Towards lightweight, adaptive provenance-based intrusion detection,” 2025, *arXiv:2404.14720*.
- [25] “APT Notes,” 2019. [Online]. Available: <https://github.com/kbandla/APTnotes>
- [26] “TC,” 2023. [Online]. Available: <https://github.com/darpa-i2o/Transparent-Computing>
- [27] “OpTC,” 2024. [Online]. Available: <https://github.com/FiveDirections/OpTC-data>
- [28] “ASAL,” 2023. [Online]. Available: <https://github.com/PKU-ASAL/Simulated-Data>
- [29] M. Zipperle, F. Gottwalt, E. Chang, and T. Dillon, “Provenance-based intrusion detection systems: A survey,” *ACM Comput. Surv.*, vol. 55, no. 7, pp. 1–36, 2022.
- [30] Z. Li, Q. A. Chen, R. Yang, Y. Chen, and W. Ruan, “Threat detection and investigation with system-level provenance graphs: A survey,” *Comput. Secur.*, vol. 106, 2021, Art. no. 102282.
- [31] Q. Wang et al., “You are what you do: Hunting stealthy malware via data provenance analysis,” 2020.
- [32] S. M. Milajerdi, B. Eshete, R. Gjomemo, and V. Venkatakrishnan, “POIROT: Aligning attack behavior with kernel audit records for cyber threat hunting,” *ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 1795–1812.
- [33] A. Goyal, X. Han, G. Wang, and A. Bates, “Sometimes, you aren’t what you do: Mimicry attacks against provenance graph host intrusion detection systems,” in *Proc. 30th Netw. Distrib. Syst. Secur. Symp.*, 2023.
- [34] S. Ruder, “An overview of gradient descent optimization algorithms,” 2016, *arXiv:1609.04747*.
- [35] H. Robbins and S. Monro, “A stochastic approximation method,” *Ann. Math. Statist.*, vol. 22, pp. 400–407, 1951.
- [36] J. Duchi, E. Hazan, and Y. Singer, “Adaptive subgradient methods for online learning and stochastic optimization,” *J. Mach. Learn. Res.*, vol. 12, no. 7, pp. 2121–2159, 2011.
- [37] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” 2014, *arXiv:1412.6980*.
- [38] J. R. Martins and A. Ning, *Engineering Design Optimization*. Cambridge, U.K.: Cambridge Univ. Press, 2021.

- [39] X. Zhang et al., "AOC-IDS: Autonomous online framework with contrastive learning for intrusion detection," in *Proc. IEEE Conf. Comput. Commun.*, 2024, pp. 581–590.
- [40] Y. Wei, C. Cheng, and G. Xie, "OFIDS : Online learning-enabled and fingerprint-based intrusion detection system in controller area networks," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 6, pp. 4607–4620, Nov./Dec. 2023.
- [41] A. Aldribi, I. Traoré, B. Moa, and O. Nwamuo, "Hypervisor-based cloud intrusion detection through online multivariate statistical change tracking," *Comput. Secur.*, vol. 88, 2020, Art. no. 101646. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167404819301907>
- [42] S. Han et al., "Log-based anomaly detection with robust feature extraction and online learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 2300–2311, 2021.
- [43] S. Arzt et al., "FlowDroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for android apps," *ACM Sigplan Notices*, vol. 49, no. 6, pp. 259–269, 2014.
- [44] S. Ma, X. Zhang, and D. Xu, "PROTRACER: Towards practical provenance tracing by alternating between logging and tainting," in *Proc. 23rd Annu. Netw. Distrib. Syst. Secur. Symp.*, 2016.
- [45] K. H. Lee, X. Zhang, and D. Xu, "High accuracy attack provenance via binary-based execution partition," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2013, vol. 16.
- [46] J. Park, D. Nguyen, and R. Sandhu, "A provenance-based access control model," in *Proc. 10th Annu. Int. Conf. Privacy, Secur. Trust*, 2012, pp. 137–144.
- [47] S. S. Khan and M. G. Madden, "One-class classification: Taxonomy of study and review of techniques," *Knowl. Eng. Rev.*, vol. 29, no. 3, pp. 345–374, 2014.
- [48] D. M. Tax and R. P. Duin, "Data domain description using support vectors," in *Proc. Eur. Symp. Artif. Neural Netw.*, 1999, vol. 99, pp. 251–256.
- [49] D. M. Tax and R. P. Duin, "Support vector domain description," *Pattern Recognit. Lett.*, vol. 20, no. 11/13, pp. 1191–1199, 1999.
- [50] Y. Nesterov, "A method for unconstrained convex minimization problem with the rate of convergence $O(1/k^2)$," in *Proc. Dokl. Akad. Nauk. SSSR*, 1983, vol. 269, no. 3, p. 543.
- [51] M. D. Zeiler, "ADADelta: An adaptive learning rate method," 2012, *arXiv:1212.5701*.



Zhenyuan Li received the PhD degree in cyber security from Zhejiang University, Hangzhou, China. He is currently an assistant professor with the College of Science and Technology, Zhejiang University. His research interests include system security and cyberattacks, intrusion detection, and cyber threat analysis.



and optimizing endpoint detection and response (EDR) systems, and host intrusion detection systems (HIDS) systems to detect and defend against advanced persistent threats (APT).

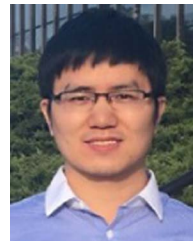
Lingzhi Wang received the BE degree in electronic information science and technology from Tsinghua University, Beijing, China, in 2020. He is currently working toward the PhD degree with Northwestern University, Evanston, IL, USA. His research interests include behavior analysis of software, AI security, and the development of the knowledge base to enhance our understanding of cyberattack techniques to improve system security, and protect against evolving threats in the digital space, and focuses on system security and cyberattacks, especially on developing



Zhengkai Wang received the BE degree in network engineering from the University of Electronic Science and Technology of China, Sichuan, China, in 2024. He is currently working toward the master's degree with Zhejiang University, Zhejiang, China. His research interests include intrusion detection and defending against advanced persistent threats (APT).



Xiangmin Shen received the BS degree in computer science from Northwestern University, Evanston, IL, USA, in 2019. He is currently working toward the PhD degree with Northwestern University, Evanston, IL, USA. His research interests include intrusion detection and security measurement, and verification.



Haitao Xu is currently an assistant professor with the School of Cyber Science and Technology, Zhejiang University, Zhejiang, China. Before joining Zhejiang University, he was a tenure-track Assistant Professor with Arizona State University. His research interests include cyber deception defense, IoT security, web security, and e-crime investigation. His work on underground cybercrime investigation was the recipient of the Best Paper nomination in WWW 2015.



Yan Chen (Fellow, IEEE) received the PhD degree in computer science from the University of California, Berkeley, CA, USA, in 2003. He is currently a professor with the Department of Electrical Engineering and Computer Science, Northwestern University, Evanston, IL, USA. His papers have been cited more than 7000 times based on Google Scholar and his H-Index is 34. His research interests include network security, measurement, and diagnosis for large-scale networks and distributed systems. He was the recipient of the Department of Energy (DoE) Early CAREER Award in 2005, the Department of Defense (DoD) Young Investigator Award in 2007, and the Best Paper nomination in ACM SIGCOMM 2010.



Shouling Ji (Member, IEEE) received the PhD degree in electrical and computer engineering from the Georgia Institute of Technology, Atlanta, GA, USA, and the PhD degree in computer science from Georgia State University, Atlanta, GA, USA. He was a research intern with the IBM T. J. Watson Research Center. He is currently a Qiushi distinguished professor with the College of Computer Science and Technology, Zhejiang University, Zhejiang, China. His research interests include data-driven security and privacy, AI security, and software and system security. He was the recipient of the 2012 Chinese Government Award for Outstanding Self-Financed Students Abroad and 10 Best/Outstanding Paper Awards, including ACM CCS 2021. He is also a member of ACM, and a senior member of CCF.